

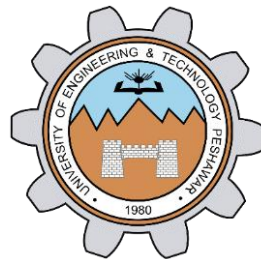
AdaptiShield: A Security Gateway and Live Monitoring Dashboard for Protecting AI Agents Against Prompt Injection Attacks

By

Muhammad Ahmad Khan (Reg NO: 23jzbc0238)

Aleena Khan (Reg NO: 23jzbc0229)

Amna Noor (Reg NO: 23jzbc0230)



Submitted to the Department of Computer Science & Information Technology

In Partial Fulfillment of the Requirements for the Degree of

**Bachelor of Science in Computer Science at the
University of Engineering & Technology, Peshawar Sep 2026 © 2026**

Supervisor: Abdullah Burhan

**University of Engineering & Technology, Peshawar. All rights
reserved.**

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
LLM	Large Language Model
IPI	Indirect Prompt Injection
PI	Prompt Injection
MCP	Model Context Protocol
API	Application Programming Interface
ASR	Attack Success Rate
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
FPR	False Positive Rate
TPR	True Positive Rate / Detection Rate
F1	F1 Score
WebSocket	WebSocket communication protocol
UI	User Interface
DB	Database
RAG	Retrieval-Augmented Generation
OWASP	Open Worldwide Application Security Project
IEEE	Institute of Electrical and Electronics Engineers
FYP	Final Year Project

Abstract

Today, AI agents can read emails, tickets and documents and then take action on them by sending messages, forwarding files or calling external services. Since the user request comes in the same context as the untrusted content, an intruder can place an instruction in a normal document, and the agent might execute it. This is referred to as indirect prompt injection. AdaptiShield is a security gateway that checks each change the agent suggests prior to the execution of any tool. It will run the agent decision twice, one time with the suspicious content visible, and one time with the suspicious content hidden or cleaned up, and consider any difference in the action chosen as an evidence of injection. The injected actions are denied, or injected text is removed and the user task continues to run. A dashboard updates with all layers decisions in real time, and the changes to the configuration must be approved by an administrator. The system will be tested on the InjecAgent and AgentDojo benchmarks and it will report the detection rate, false alarm rate and task completion rate by attack type.

Introduction

Large Language Models are not just for answering questions. They're used as agents: Systems that read from external sources and then call tools to fulfill a user's request. An assistant can read inbox, reply, read a support ticket and update a record, or read a build log and open a PR. It's commonplace to attach a model to file systems, mail servers, internal APIs, with protocols like the Model Context Protocol.

That connection will be a security issue that the model cannot resolve. The user instruction and what the agent needs to read, both are in the same context window and in the same form, there's no way for the model to tell which part is data it should process and which is a command it should follow. If the attacker is able to write a document the agent will read, the attacker can put a line in it, like the end of a meeting email, requesting that the agent forward the thread to an outside email address. The attacker does not touch the user prompt and never interacts with the model directly. This attack is called indirect prompt injection; the first description of it is by Greshake et al. [1]. It is now at the No.1 spot in the OWASP list of risks to LLM applications

Two families of defense are common. Prompt level defenses mark the untrusted span and tell the model to ignore commands inside it. Gate level defense controls are used to further limit what the agent can do after with permission scopes, egress allowlists and tool filters. Both make inferences about surface: one is based on the look of the text, one is based on the action. Neither asks whether the untrusted text was what was responsible for the decision.

Motivation

This attack does not only result in an incorrect answer. An agent that has access to mail can disclose an entire thread. An agent that has access to a file may delete or upload data. A permission can be modified by an agent having an internal API key. Published benchmarks demonstrate exposure is not hypothetical: a prompted GPT-4 agent followed injected instructions in about a 25% of instances, while weaker open models performed significantly worse [2] **Invalid source specified..** According to AgentDojo, prompt level defenses will make attacks less likely, but not impossible, and that a significant percentage of targeted attacks will still pass through. [3]

Organizations that have implemented agents are also unable to see what is going on. When an action is blocked, no one is able to tell the reason. If the attack succeeds, it typically shows up after the damage. This is exacerbated by an automatic defense tuning, which can change a defense without anyone looking at what the evidence is to ensure it meets all requirements.

Let's solve this for two reasons. This is because the question is causal rather than about the wording of the attack, so rephrasing the payload cannot evade it. Secondly, you can't trust a defense that you can't see. The detection engine itself was developed and tested in our own previous studies, and the most important thing that we learned from our previous studies was not just the accuracy but, how much of the attacks were detected, how much were missed and why. In this project, that engine becomes a system which can be viewed, tested, and supervised by a person.

Methodology

The system is designed as 5 modules that share common backend.

Protected demo agent. Small helper for email and documents that reads and helps to send, forward, delete and upload documents. It provides a realistic target for the project, so that an injected email can be shown end-to-end.

Attack Lab. A tester picks an attack from the benchmark collections or computes an attack himself, and executes the attack with protection in both on and off mode, and then compares the results.

Defense engine. All tools proposed are used through it prior to execution. It marks content as trusted or untrusted, filters answers to tools, enforces static policy rules and forwards high impact actions to the causal check. The causal check executes the action selection through various perspectives of the input (with the suspicious span shown or hidden/sanitized), and compares the outcomes. If the untrusted span makes a changed decision, this is considered as evidence that it was responsible for the action. The engine will block the action or take away the text injected and then allow the user task(s) to proceed, and it places limits on the permissions and destination before running any tool.

Live Defense Monitor. A visualization to a web dashboard streamed as each layer decision is made, detailing what was flagged, what the causal check returned, which rule fired and the final status as a request is processed via WebSockets. A replay feature replays recorded episodes to instantaneously demonstrate results.

Admin console and analytics. The changes to the configuration that the adaptive component wants are reviewed by an administrator and the supporting evidence is recalculated before being accepted or rejected. Analytics pages show detection, false alarm and task completion rates over time per attack type.

Datasets. The use of two public benchmarks. The external attack corpus is InjecAgent [2] which provides tool integrated attack cases. AgentDojo [3] provides benign documents to measure how frequently a defense raises a false alarm on benign documents, as well as a held out attack set. A small set of attacks written by us are used for development purposes only, and results on this set are reported separately from results on external corpora.

Tools. Backend: Python / FastAPI / WebSockets. The frontend is React with Vite, Tailwind CSS and Recharts. The models are run locally using Ollama with Gemma 3 4B for causal probe and Qwen 2.5 3B for sanitisation and planning, so it doesn't require a paid cloud API and requires a 4 GB GPU. Episodes, verdicts and approval history are stored in PostgreSQL. Packaging, version control and automated testing are covered by Docker, GitHub, and Pytest. Spotlighting [5] is re-implemented as a comparator arm in the same pipeline.

Evaluation. The same corpora are used to measure the attack success rate, the workflow continuation rate, and false positive rate for an undefended baseline, a prompt level baseline and the full system. Reports are not presented by attacking type as one overall number; a overall number would mask the point at which a defense fails.

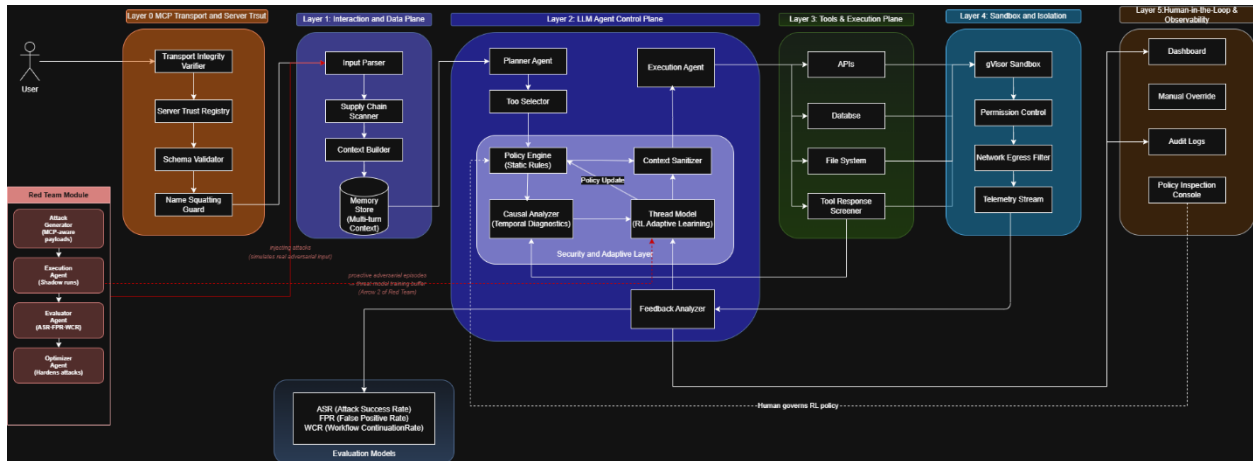
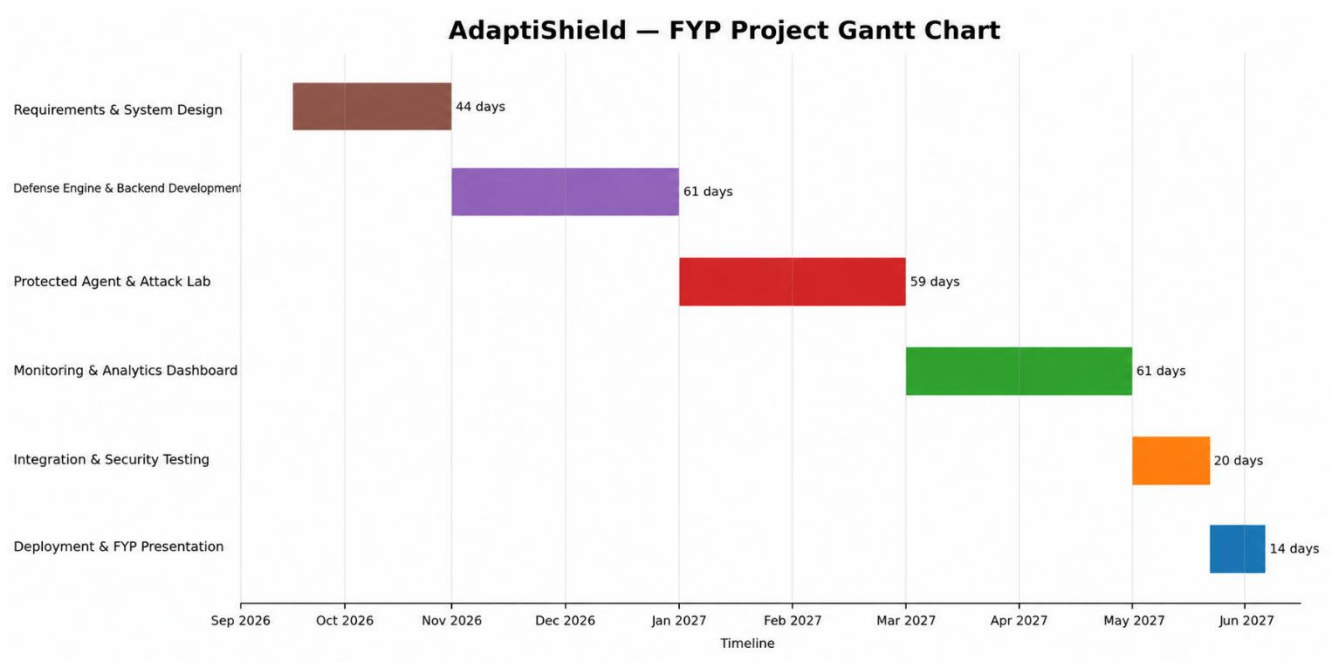


Figure: System Architecture



Graph: Gantt Chart

Conclusion

We will have a working security gateway that will defend a real AI agent from instructions embedded in the content it reads, a dashboard that will explain every allow, block or safe continuation in real time, a review console that will ensure that a person is in control of the automatic configuration changes, and measured results of the two public benchmarks broken down by the type of attack, including the categories where the detection is weak. The output is a fully

functional prototype that is deployable, with documentation and a test suite, all executed on local open source models.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You Have Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” arXiv:2302.12173, 2023.
- [2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Prompt Injection Attacks and Defenses in LLM-Integrated Applications,” arXiv:2310.12815, 2023.
- [3] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the Risks of LM Agents with an LM-Emulated Sandbox,” in Proc. ICLR, 2024.
- [4] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending Against Prompt Injection with Structured Queries,” in Proc. USENIX Security, 2025.
- [5] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions,” arXiv:2404.13208, 2024.
- [6] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, “Defending Against Indirect Prompt Injection Attacks With Spotlighting,” arXiv:2403.14720, 2024.
- [7] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in Findings of ACL, pp. 10471–10506, 2024.
- [8] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.
- [9] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents,” in Proc. ICLR, 2025.
- [10] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks, “HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal,” in Proc. ICML, 2024.

- [11] S. Toyer, O. Watkins, E. A. Mendes, J. Svegliato, L. Bailey, T. Wang, I. Ong, K. Elmaaroufi, P. Abbeel, T. Darrell, A. Ritter, and S. Russell, “Tensor Trust: Interpretable Prompt Injection Attacks from an Online Game,” arXiv:2311.01011, 2023.
- [12] T. Wu, S. Zhang, K. Song, S. Xu, S. Zhao, R. Agrawal, S. R. Indurthi, C. Xiang, P. Mittal, and W. Zhou, “Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy,” arXiv:2410.09102, 2024.
- [13] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating Prompt Injections by Design,” arXiv:2503.18813, 2025.
- [14] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, Y. Gal, and X. Davies, “AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents,” in Proc. ICLR, 2025.
- [15] OWASP Foundation, “OWASP Top 10 for LLM Applications 2025,” OWASP GenAI Security Project, 2025.
- [16] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, “WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 38, 2025.
- [17] S. J. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan, “Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks,” arXiv:2512.06556, 2025.
- [18] S. M. A. Hossain, R. K. Shayoni, M. R. Ameen, A. Islam, M. F. Mridha, and J. Shin, “A Multi-Agent LLM Defense Pipeline Against Prompt Injection Attacks,” arXiv:2509.14285, 2025.
- [19] Z. Wang, N. Nagaraja, L. Zhang, H. Bahsi, P. Patil, and P. Liu, “To Protect the LLM Agent Against the Prompt Injection Attack with Polymorphic Prompt,” arXiv:2506.05739, 2025.
- [20] OWASP Foundation, “LLM01:2025 Prompt Injection,” OWASP GenAI Security Project, 2025.

Supervisor Approval:

Name: _____



Signature: _____